

Forms: Validation and FormData

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

A Form to Work With

- Every field needs a **name**; that is the key JS and the server use

```
<form id="signup" novalidate>
  <input type="text" name="username">
  <input type="email" name="email">
  <input type="number" name="age">
  <label><input type="checkbox" name="newsletter"> Newsletter</label>
  <label><input type="radio" name="plan" value="free" checked> Free</label>
  <label><input type="radio" name="plan" value="pro"> Pro</label>
  <select name="country">
    <option value="us">United States</option>
    <option value="uk">United Kingdom</option>
  </select>
  <button type="submit">Create account</button>
</form>
```

Reaching the Form and Fields

- Grab the form, then reach fields by `name` via its `elements` collection

```
const form = document.getElementById("signup");

const username = form.elements["username"];
const email = form.elements["email"];

console.log(username.name);    // "username"
console.log(username.tagName); // "INPUT"
```

- Cleaner than a `querySelector` for every input

Reading Field Values

```
// Text / email / password / textarea - always a STRING
form.elements["username"].value.trim();
Number(form.elements["age"].value);      // "42" -> 42

// Checkbox: use .checked, NOT .value
form.elements["newsletter"].checked;     // true / false

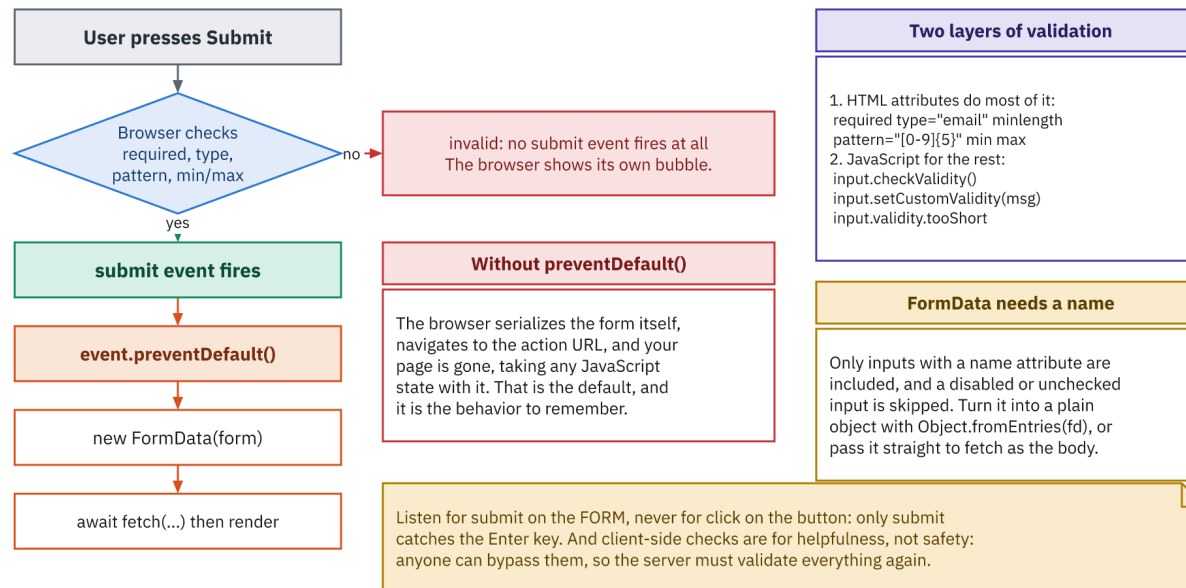
// Radio group: the group's .value is the selected one
form.elements["plan"].value;              // "free" or "pro"

// Select: .value is the chosen option
form.elements["country"].value;          // "us"
```

The `submit` Event

The Form Submit Lifecycle

The browser will navigate away unless you stop it. Everything else hangs off that one decision.



- Listen on the **form**: only `submit` catches the Enter key
- Without `preventDefault()` the browser navigates away and your page is gone

Validation Is UX, Not Security

- Code that checks input **before** you trust it
- Catches blanks, an email with no @, letters in a phone number
- A determined user can bypass anything in the browser
- **Always re-validate on the server**; client-side is for the friendly path

Validating Without Regex First

- Many rules are just plain JavaScript

```
const username = form.elements["username"].value.trim();
if (username === "") console.log("Required.");
if (username.length < 3) console.log("Too short.");

const age = Number(form.elements["age"].value);
if (Number.isNaN(age) || age < 18) console.log("Must be 18+.");

const pw = form.elements["password"].value;
const confirm = form.elements["confirm"].value;
if (pw !== confirm) console.log("Passwords do not match.");
```

- Use regex only when the rule is about the **shape** of a string

Creating a Regex and `test()`

```
const zip = /^\\d{5}$/;           // literal - preferred
const dynamic = new RegExp(word, "i"); // when it comes from a variable

zip.test("90210");    // true
zip.test("9021");     // false (only 4 digits)
zip.test("90210-1");  // false (extra chars)

"Order #4821 shipped".match(/\\d+\\/)[0]; // "4821"
```

- `regex.test(string)` → boolean, which is exactly what validation wants

Reading the Pattern Syntax

Anatomy of a Regular Expression

One pattern, every part labeled. Read it left to right, the way the engine does.

`/^[A-Z]{2}-\d{3,4}$/i`

^ Anchor: must match from the very START of the string.	[A-Z] Character class: any ONE character from the set.	{2} Quantifier: exactly two of the thing before it.	- A literal hyphen. Most characters just mean themselves.
\d Shorthand for [0-9]. Also \w word and \s space; \D \W \S negate.	{3,4} Between three and four. Also * zero or more, + one or more, ? optional.	\$ Anchor: must reach the END. ^ and \$ together mean the WHOLE string must match.	i A flag, after the closing slash. i ignore case, g find all, m multiline.
Matches: AB-123 · ab-4567 · ZZ-999		Does not: A-123 · AB-12 · AB-12345 · xAB-123	

Use a regex to check the SHAPE of a short string. Do not use one to parse HTML, and do not invent your own for email; type="email" already does it. A pattern with a g flag keeps lastIndex between calls, so reusing one across a loop can skip matches; make a fresh regex, or use String.matchAll.

- Also: `( ... )` groups, `|` means "or", and `\.` is a literal dot
- `^...$` forces the **whole** string to match, usually what you want

Common Validation Patterns

```
// Email - practical, good-enough
const email = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

// US phone - optional pieces
const phone = /^(?\\d{3}\\)?[-.\\s]?\\d{3}[-.\\s]?\\d{4}$/;

// US ZIP - 5 digits, optional +4
const zip = /^\\d{5}(-\\d{4})?$/;

// Password: lookaheads assert "somewhere ahead there is..."
const strong = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\\d){8,}$/;
strong.test("Password1"); // true
strong.test("password");  // false
```

Showing Inline Errors

- Fill a per-field container; toggle an `invalid` class

```
function showError(field, message) {  
    document.getElementById(field + "-error").textContent = message;  
    form.elements[field].classList.add("invalid");  
}  
  
function clearError(field) {  
    document.getElementById(field + "-error").textContent = "";  
    form.elements[field].classList.remove("invalid");  
}
```

Validating on Submit and on Input

- Check **every** field so the user sees all errors at once

```
form.addEventListener("submit", (event) => {  
  event.preventDefault();  
  let valid = true;  
  
  const email = form.elements["email"].value.trim();  
  if (!emailPattern.test(email)) {  
    showError("email", "Enter a valid email.");  
    valid = false;  
  } else clearError("email");  
  
  if (valid) form.reset();  
});  
  
// Live feedback as they type:  
form.elements["email"].addEventListener("input", (e) => { /* ... */ });
```

novalidate and the Constraint Validation API

- Without `novalidate`, the browser blocks submit and your handler **never runs**

```
<input type="email" name="email" required minlength="8" pattern="[a-z]+">
```

```
email.checkValidity();           // true / false
email.validity.valueMissing;     // required unmet
email.validity.typeMismatch;     // type="email" not satisfied
email.validity.patternMismatch;  // pattern failed
email.validationMessage;        // the browser's own text

confirm.setCustomValidity("Passwords do not match."); // "" clears it
```

- Let HTML hold the ordinary rules; use JS for the rest

FormData: Every Field in One Call

- `new FormData(form)` scoops up **every named field**, files included
- Fields with no `name` are ignored; unchecked checkboxes are skipped

```
const data = new FormData(form);

data.get("username"); // "ada"
data.get("missing");  // null

// A repeated name - a checkbox group or <select multiple>:
data.getAll("topics"); // ["js", "css"]
data.get("topics");     // "js" (first only)

for (const [key, value] of data.entries()) console.log(key, value);
```

FormData: Convert and Send

```
data.append("source", "web"); // add; set() replaces instead

const obj = Object.fromEntries(data.entries());
obj.topics = data.getAll("topics"); // fromEntries keeps only the LAST
```

- Send it two ways:

```
// multipart - pass FormData straight through (this carries files)
await fetch("/signup", { method: "POST", body: data });

// JSON - stringify the object (no files)
await fetch("/signup", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(obj),
});
```